

Curso <i>Engenharia de Software</i>	Campus <i>Rio Verde</i>
<i>Programação Orientada a Objetos</i>	
Nome do(a) acadêmico(a)	Valor
Professor <i>Sergio Souza Novak</i>	Turma <i>B</i>
Data	Código <i>ESW448</i>

QUESTÕES

1. **Logística Multimodal — Pátio de Caminhões:** Em uma plataforma de logística multimodal, é necessário modelar um sistema para gerenciar a entrada, saída e permanência de caminhões em um pátio de triagem. Observe o diagrama da classe **PatioCaminhoes** abaixo e implemente-a em Java, respeitando todas as regras de negócio descritas a seguir.

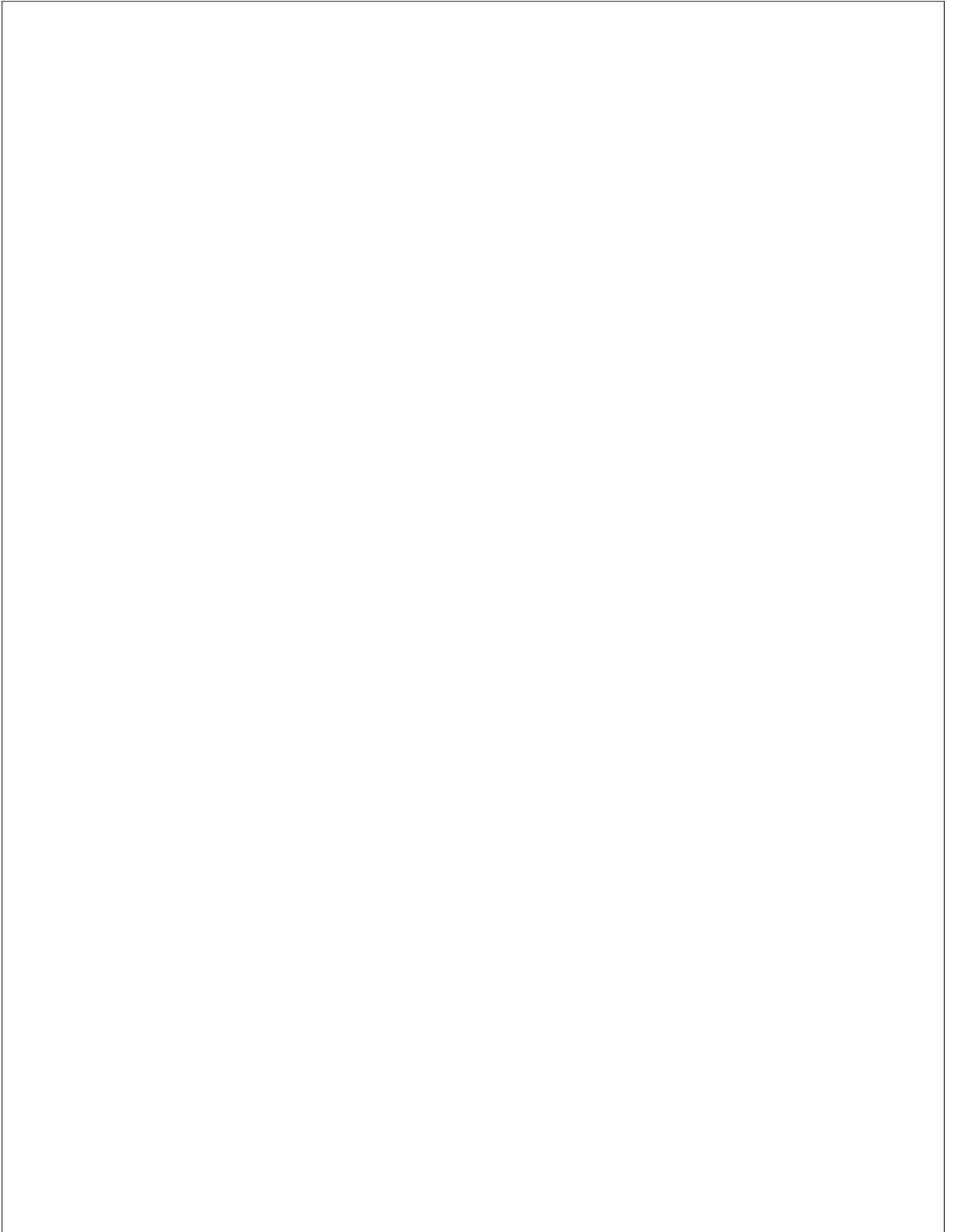
PatioCaminhoes
- nome : String - capacidadeMaxima : int - caminhoesEstacionados : int - tempoMedioEspera : double - aberto : boolean - totalCaminhoesAtendidos : int + PatioCaminhoes(nome: String, capMax: int) + abrirPatio() : void + fecharPatio() : void + receberCaminhao(qtd: int) : boolean + liberarCaminhao(qtd: int) : boolean + getOcupacaoAtual() : int + patioLotado() : boolean - atualizarTempoEspera(qtd: int) : void - verificarCapacidade(qtd: int) : boolean

Regras de negócio:

- O **construtor** recebe o nome do pátio e a capacidade máxima. O tempo médio de espera deve ser inicializado com 0.0. Os demais atributos numéricos iniciam com seus valores padrão;
- `abrirPatio()` e `fecharPatio()` alteram o estado do atributo **aberto**;
- `receberCaminhao(qtd)`: só pode receber caminhões se o pátio estiver **aberto** e se houver capacidade disponível (verificada pelo método privado `verificarCapacidade`). Se aceito, incrementa `caminhoesEstacionados` e `totalCaminhoesAtendidos` pela quantidade recebida, e chama o método privado `atualizarTempoEspera`. Retorna `true` se a operação for realizada, `false` caso contrário;

- `liberarCaminhao(qtd)`: só permite liberar se a quantidade solicitada **não ultrapassar** o número de caminhões estacionados. Se permitido, decrementa `caminhoesEstacionados`. Retorna `true` ou `false`;
- `getOcupacaoAtual()`: retorna o número de caminhões estacionados;
- `patioLotado()`: retorna `true` se `caminhoesEstacionados >= capacidadeMaxima`;
- `atualizarTempoEspera(qtd)` (**privado**): incrementa o tempo médio de espera em `qtd * 0.5`;
- `verificarCapacidade(qtd)` (**privado**): retorna `true` se `(caminhoesEstacionados + qtd) <= capacidadeMaxima`.

Implemente também uma classe `Main` com o método `main`, criando um pátio chamado "Terminal Sul" com capacidade para 200 caminhões, abrindo o pátio e recebendo 50 caminhões.



2. **Agronegócio — Silo de Armazenamento (Sementeira Comigo):** No contexto de sistemas de gerenciamento agroindustrial da cooperativa Comigo, é necessário modelar um silo de grãos para controlar a entrada e saída de cargas, bem como as condições de umidade interna. Observe o diagrama da classe **SiloArmazenamento** abaixo e implemente-a em Java, respeitando todas as regras de negócio descritas a seguir.

SiloArmazenamento
<pre>- identificador : String - capacidadeToneladas : double - volumeAtual : double - umidadeInterna : double - umidadeMaxima : double - ativo : boolean - totalDescarregado : double + SiloArmazenamento(id: String, capTon: double, uMax: double) + ativarSilo() : void + desativarSilo() : void + receberCarga(toneladas: double) : boolean + expedirCarga(toneladas: double) : boolean + getVolumeAtual() : double + siloEmAlerta() : boolean - controlarUmidade(toneladas: double) : void - verificarEspaco(toneladas: double) : boolean</pre>

Regras de negócio:

- O **construtor** recebe o identificador do silo, a capacidade em toneladas e a umidade máxima permitida. A umidade interna deve ser inicializada com 10.0 (umidade ambiente controlada). Os demais atributos numéricos iniciam com seus valores padrão;
- `ativarSilo()` e `desativarSilo()` alteram o estado do atributo `ativo`;
- `receberCarga(toneladas)`: só pode receber carga se o silo estiver **ativo** e se houver espaço disponível (verificado pelo método privado `verificarEspaco`). Se aceito, incrementa `volumeAtual` e `totalDescarregado` pelas toneladas recebidas, e chama o método privado `controlarUmidade`. Retorna `true` se a operação for realizada, `false` caso contrário;
- `expedirCarga(toneladas)`: só permite expedir se a quantidade solicitada **não ultrapassar** o volume atualmente armazenado. Se permitido, decrementa `volumeAtual`. Retorna `true` ou `false`;
- `getVolumeAtual()`: retorna o volume atualmente armazenado no silo;
- `siloEmAlerta()`: retorna `true` se `umidadeInterna > umidadeMaxima`;
- `controlarUmidade(toneladas)` (**privado**): incrementa a umidade interna em toneladas * 0.3. Se a umidade interna ultrapassar a umidade máxima, o silo deve ser **desativado automaticamente**;
- `verificarEspaco(toneladas)` (**privado**): retorna `true` se `(volumeAtual + toneladas) <= capacidadeToneladas`.

Implemente também uma classe `Main` com o método `main`, criando um silo com identificador "SILO-07", capacidade de 5000.0 toneladas e umidade máxima de 60.0. Ative o silo e receba uma carga de 1200.0 toneladas, exibindo o volume atual e se o silo está em alerta.

